

Practical : 10	Implementation of Kruskal's Algorithm for Minimum Spanning Tree.
Date : 12/08/2026	

Objective :- To implement Kruskal's Algorithm in C++ using edge-list sorting and a Union-Find (Disjoint Set Union) structure to construct the Minimum Spanning Tree (MST) of a weighted, connected, undirected graph, and to analyse its time and space complexity.

Program :-

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Edge { int u, v, w; };

struct DSU {
vector<int> parent, rnk;
DSU(int n) : parent(n), rnk(n, 0) {
for (int i = 0; i < n; i++) parent[i] = i;
}
int find(int x) {
if (parent[x] != x) parent[x] = find(parent[x]);
return parent[x];
}
bool unite(int x, int y) {
int rx = find(x), ry = find(y);
if (rx == ry) return false;
if (rnk[rx] < rnk[ry]) swap(rx, ry);
parent[ry] = rx;
if (rnk[rx] == rnk[ry]) rnk[rx]++;
return true;
}
};

// ----- Kruskal's Algorithm (Sort + Union-Find, O(E log E)) -----
int kruskalMST(int V, vector<Edge>& edges, vector<Edge>& mstEdges) {
sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
return a.w < b.w;
});
DSU dsu(V);
int totalWeight = 0;
for (auto& e : edges) {
if (dsu.unite(e.u, e.v)) {
mstEdges.push_back(e);
totalWeight += e.w;
}
}
return totalWeight;
}
```

```

}

int main() {
int V = 5;
vector<Edge> edges = {
{0,1,2}, {0,3,6}, {1,2,3}, {1,3,8}, {1,4,5},
{2,4,7}, {3,4,9}
};

vector<Edge> mstEdges;
int totalWeight = kruskalMST(V, edges, mstEdges);

cout << "Edge \tWeight\n";
for (auto& e : mstEdges)
cout << e.u << " - " << e.v << "\t" << e.w << "\n";

cout << "Total weight of MST: " << totalWeight << endl;
return 0;
}

```

Output :-

```

Edge      Weight
0 - 1      2
1 - 2      3
1 - 4      5
0 - 3      6
Total weight of MST: 16

```

Analysis:-

Time & Space Complexity (Sort + Union-Find implementation, V = vertices, E = edges):

Method	Time Complexity	Space Complexity
Kruskal's - Sort + Union-Find	$O(E \log E)$	$O(V + E)$
Kruskal's - Union-Find (path comp. + rank)	$O(E \alpha(V))$ per op.	$O(V)$

Kruskal's algorithm builds the MST by greedily picking the globally cheapest edge that does not create a cycle, using a Disjoint Set Union (DSU) structure to test connectivity in near-constant time. Sorting the E edges dominates the running time at $O(E \log E)$; after sorting, each of the E edges is processed with a find/union operation that runs in $O(\alpha(V))$ amortised time using path compression and union by rank, so the union-find phase itself costs only $O(E \alpha(V))$, which is effectively linear. This makes Kruskal's a good fit for sparse graphs represented as an edge list, in contrast to Prim's adjacency-matrix approach which prefers dense graphs. On the same 5-vertex sample graph used for Prim's algorithm, Kruskal's algorithm selects edges (0-1, 1-2, 1-4, 0-3) in increasing weight order and arrives at an identical MST of total weight 16, confirming that both algorithms converge on the same minimum spanning tree when one exists.

Conclusion :- Kruskal's Algorithm was successfully implemented in C++ using an edge list sorted by weight combined with a Union-Find (Disjoint Set Union) structure to detect cycles. The program correctly built the Minimum Spanning Tree and total weight for the sample graph, matching the result obtained from Prim's algorithm and confirming the $O(E \log E)$ time complexity dominated by the sorting step. Kruskal's algorithm is particularly well suited to sparse graphs stored as edge lists, while Prim's algorithm with an adjacency matrix is preferable for dense graphs.